

WEEKLY DIARY CONTENT

Industry Internship & Project (IIP)

Mr. Prathamesh Vijay Jadhav | PRN: 2203065

Department of Computer Science & Engineering

Cognizant Technology Solutions, Chennai

Academic Year: 2025-2026

The content below is laid out one week per page and is intended as a reference while filling in a handwritten weekly diary. Each week contains four sections matching the diary template, with line counts approximately calibrated for a standard ruled A4 book. The text is written from the candidate's perspective in flowing prose, with technical detail drawn from the Cognizant Gen C Salesforce curriculum and the University Management System project.

Week 1

Delta Phase - Foundations and Behavioural Training

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

This week introduced the Software Development Life Cycle in detail, with separate sessions on the Waterfall approach and on Agile-Scrum. We covered sprint mechanics, user stories and story-point estimation. The behavioural-training module on communication, body language and e-mail etiquette ran in parallel. On the equipment side, my Cognizant laptop, VPN access and corporate ID were set up and tested.

2. Project Work Progress *(about 25 lines)*

There was no active project work this week because the engagement opened with the delta phase covering foundations. Most of the week was occupied by orientation, HR onboarding and a structured walk-through of the sixty-day Gen C curriculum, the milestones, the hackathon phases and the rapid holistic assessments. The trainer compared the Waterfall and Agile-Scrum models using real-life client situations, and we attempted a mock exercise where we picked a fictional problem and decomposed it into user stories. We also discussed estimation in story points and the meaning of definition of done. The project-management module covered Gantt charts and the roles of a Scrum Master and a Product Owner. I asked my industry mentor about ongoing live projects in the Salesforce practice to better understand the kind of work the team handles. By the end of the week I had a clearer picture of how Cognizant approaches client delivery and what the expectations are for a Gen C intern. I drew up a personal weekly target sheet for the rest of the programme, listing what I want to cover and the small extras I want to do beyond the curriculum.

3. Challenges Faced *(about 12 lines)*

Switching from a college learning pace to a corporate training pace was sharper than I had expected. The biggest difficulty was the volume of new terminology - sprint, backlog, kanban, increment, product owner, story point - introduced in rapid succession during the first sessions. Daily stand-ups also took some getting used to since I had not earlier been asked to summarise my work in a strict one-minute window. Sitting through long classroom sessions without phone access required some adjustment. Network connectivity in the training room was unstable for the first two days, which slowed down the Trailhead account setup steps.

4. Additional Comments *(about 15 lines)*

The first week genuinely felt like the start of something new. The Cognizant Chennai campus is much larger than the academic spaces I am used to, and the cafeteria, library and quiet rooms were all impressive. I met about thirty fellow Gen Cs from various engineering colleges across India, and it has been interesting to observe how varied our backgrounds are despite all of us having a Computer Science foundation. The trainers are senior Salesforce subject-matter experts and they managed to keep the otherwise dry foundational material engaging with stories from actual client engagements. I noted some of those examples in a side notebook to revisit later. By the end of the week I realised this is going to be a marathon rather than a sprint, and that pacing myself rather than over-exerting in the first month will pay off in the long run.

Week 2

Delta Phase - Web Foundations and Data Modelling

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

This week was devoted to web foundations and conceptual data modelling. Sessions covered HTML structure, the CSS box model and selectors, JavaScript basics including the DOM, JSON object representation and array handling. The data-modelling block introduced conceptual versus logical models, normalisation up to third normal form, and entity-relationship diagrams. A foundational assessment at the end measured how well we had picked up the delta material.

2. Project Work Progress *(about 25 lines)*

Project work was still limited to small exercises rather than the actual UMS build. Through the week we built a series of mini HTML pages styled with plain CSS - a personal portfolio, a small registration form and a single-page contact card. We then layered in JavaScript to handle simple validations and to populate a part of the page from a static JSON object. The trainer walked us through how the document object model is structured and how event listeners attach to nodes. Several short hands-on exercises were given, including building a to-do list that stored items in memory and rendered them dynamically. On the data-modelling side, I drew an entity-relationship diagram for a small library management scenario and then normalised the schema, identifying which attributes should sit on which entity and where junction tables would be required. I also spent some personal time thinking about what a data model for a university management system might look like, sketching tables for Departments, Courses, Students, Applications, Faculty and Attendance. By the end of the week I had a rough conceptual data model with about eight entities and the key relationships between them.

3. Challenges Faced *(about 12 lines)*

The biggest challenge this week was JavaScript itself. My undergraduate exposure to JavaScript had been limited and I had to brush up quickly on closures, scope and the event model. Asynchronous JavaScript through Promises and `async/await` was introduced briefly and I could see that it would require more practice before I felt confident. The entity-relationship diagram exercise was harder than I expected, particularly when normalising the schema beyond second normal form. The foundational assessment at the end of the week put pressure on us to consolidate everything we had learnt from day one, and I made a few small mistakes on JSON-related questions that I noted down.

4. Additional Comments *(about 15 lines)*

A clear realisation this week was that my college curriculum had not gone deep enough on modern JavaScript. The trainer suggested that we self-study ES6 features such as arrow functions, the spread operator and destructuring before the Lightning Web Components block, which is several weeks away but will rely heavily on those concepts. I started keeping a small JavaScript notebook with running code snippets that I refer back to during practice. The day-end recap sessions, where the trainers spend the last twenty minutes summarising what was covered, have been particularly useful for retaining material. I also joined an informal study group of three other Gen Cs who meet for half an hour each evening to clarify doubts. The mood in the batch has settled into a comfortable rhythm and I am starting to enjoy the daily routine here much more than I did in the first few days.

Week 3

Salesforce Core (Part I) - On-boarding and Org Setup

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Started the Salesforce track with on-boarding to the Cognizant training rails and induction to Trailhead and the Trailhead Playground. Learned to spawn a developer organisation and walked through organisation setup including company information, user setup, login access policies and session settings. Covered standard objects briefly and contrasted Classic with Lightning Experience. The biggest block was profiles, standard profiles and permission sets, ending in a guided workshop.

2. Project Work Progress *(about 25 lines)*

With Salesforce-specific work now beginning, project ideation moved a step forward. I spawned my personal developer org and configured the basics such as company information, fiscal-year setup, business hours and user defaults. I created a small group of test users with different profiles to feel the differences between Standard User, Read Only, System Administrator and a custom profile. I then experimented with cloning a profile and assigning it to a new test user, simulating the kind of multi-role setup a university management system would need. The training organisation also walked us through Trailhead modules on Salesforce Platform Basics, Data Modelling and User Management, and I completed three additional badges outside the assigned ones to reinforce concepts. I started sketching the user roles I would need in my own project - Student, Faculty, Admission Cell, Head of Department and a guest applicant - and considered which of these would be profiles versus permission sets. The trainer's emphasis on permission sets as the right unit of access made me reconsider an earlier instinct to put everything into custom profiles. I also began noting which standard objects I might not need and which custom objects would carry the bulk of my domain logic.

3. Challenges Faced *(about 12 lines)*

The Trailhead interface looked intimidating at first because of the sheer number of modules, trails, projects and superbades available. It took me a couple of attempts to find a good learning rhythm where I followed the Cognizant track without getting distracted by tangential modules. The conceptual difference between a profile and a permission set was confusing initially - I treated them as similar things and only realised the distinction during the workshop, where the trainer demonstrated that profiles are best kept lean while permission sets carry incremental capabilities. Session settings and login IP ranges also felt abstract until I tried logging in from a different network.

4. Additional Comments *(about 15 lines)*

Trailhead is genuinely gamified and pleasant - the points, badges and superbades create a clear sense of progression that I found motivating outside formal sessions. The trainer drew a useful analogy comparing Salesforce profiles to Linux user permissions, which made the concept click for me. I am going to keep this kind of cross-domain analogy in mind when I try to explain Salesforce to peers. By midweek I had set up two scratch orgs in addition to my main developer org and was switching between them comfortably using the SFDX CLI. The hands-on portion is now clearly more interesting to me than the classroom theory. I called home and described how the work actually involves clicking through a real interface and seeing changes immediately, which made the explanation easier than my usual abstract descriptions of college coursework.

Week 4

Salesforce Core (Part II) - Security, Custom Objects and Relationships

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Covered data security and access in depth, including organisation access, object access, record access and field-level security. Studied organisation-wide defaults, role hierarchy, sharing rules, manual sharing, permission-set groups and restriction rules. Built custom objects with various field types, dependent picklists, rollup summaries and field history tracking. Worked through page layouts, record types and the full spectrum of relationships: lookup, master-detail, self, hierarchical, many-to-many and external.

2. Project Work Progress *(about 25 lines)*

This was the first week I built objects that could one day live in my actual UMS project. I created Department and Course as a parent-child pair using a master-detail relationship, with the cascade-delete behaviour explicitly tested. I then added Application and Application Document with the same master-detail pattern. The trainer's exercises pushed me to deliberately try edge cases - converting lookup to master-detail and back, deleting a parent and observing what happens to children, and changing field types to see which conversions Salesforce permits silently. I set the organisation-wide default for Application to Private internally and externally and created a sharing rule that opens it up to the Admission Cell role. I then logged in as a test student user to confirm that they could not see other students' applications, which is essential for a real public-facing application form. Page layouts were assigned per profile so that the Admission Cell sees the full record while a Student profile sees only a curated set of fields. By the end of the week I had an evolving twelve-object data model, with five master-detail relationships, three lookups and one anticipated junction object. I am keeping a printed object schema diagram on my desk and updating it after every session.

3. Challenges Faced *(about 12 lines)*

Master-detail relationships came with several subtle behaviours that I only understood after running into them. The inability to easily revert a master-detail to a lookup after data has been created is a constraint I almost violated until the trainer warned us. Choosing between a lookup and a master-detail relationship for a borderline case took me longer than expected because both seemed defensible. The mental model around organisation-wide defaults interacting with role hierarchy and sharing rules took a full day to settle - I had to draw it out on paper twice before it made sense. Field-level security versus profile-level CRUD is another distinction I had to revisit.

4. Additional Comments *(about 15 lines)*

Salesforce's security model is more sophisticated than I had imagined, and I now understand why the platform is trusted with sensitive customer data at scale. The exercise of logging in as another user and seeing exactly what they can or cannot do is a powerful debugging tool that I expect to use throughout the project. The View All and Modify All distinction at the object level versus the record level finally clicked for me. I made a small one-page security cheat sheet for myself summarising OWD, sharing rules, role hierarchy and permission sets. The trainer's repeated message that a permission set is the right unit of access is starting to shape my design choices. I shared the cheat sheet with my study group and they were happy to copy it. Looking forward to building the first real workflows next week.

Week 5

Salesforce Core (Part III) - Automation and Reports

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Worked through Salesforce's automation tools end-to-end: validation rules, approval processes and Flow Builder with record-triggered, screen, autolaunched and scheduled flows. Covered custom formula fields, comparative trade-offs between automation tools, and the recommended use of flows in preference to the retired Process Builder. Reports and dashboards were introduced including report types, custom report types, bucket fields, subscriptions and dynamic dashboards.

2. Project Work Progress *(about 25 lines)*

This week the project began to feel real. I built validation rules on the Application object to prevent submissions where percentage fields are blank or out of range, and to enforce a status-transition guard so that a draft application cannot be moved directly to Approved without going through verification. The most interesting build was a record-triggered flow that sends an acknowledgement email to the applicant when an Application is first inserted, copying the application number into the email body via a Lightning Email Template. I also configured a second flow to push an internal notification to the Admission Cell queue when an application is marked Ready for Verification. On the reporting side, I created two simple tabular reports - applications received per department and applications by status - and pinned them to a dashboard intended for the Admission Cell. Both the report and the dashboard were configured with the right running user so that role-based filtering would work later. The trainer's emphasis on declarative-first thinking has clearly shaped how I am approaching this layer. I am keeping the validation rule logic in a shared document so that the team can review it before the hackathon next week.

3. Challenges Faced *(about 12 lines)*

Flow debugging turned out to be more tedious than I expected. The debug panel inside Flow Builder helps but the iteration cycle is slower than running a test class. I ran into a recurring issue where a record-triggered flow fired in the wrong order relative to a workflow rule I had inherited from an earlier tutorial, and tracking the order of execution took half a day. The approval process is conceptually heavier than I had assumed, with multiple steps, conditional approvers and field updates. I have also been struggling with the mental model when flows and triggers coexist on the same object.

4. Additional Comments *(about 15 lines)*

The declarative-first mindset is no longer just a slogan; it actively guides what I build and what I avoid. Several things that I would have automatically reached for code to do, I can now achieve with a record-triggered flow and a validation rule. That said, I am also looking forward to the Apex sessions in a couple of weeks because some of the trickier admission-cell workflows clearly need procedural logic with rollback support. The reports and dashboards block was lighter than I expected, but the realisation that dynamic dashboards can be filtered by the running user means I can give every role its own personalised view without building separate components. I shared my running design note with my industry mentor and he commented that the level of detail is on par with what he sees in junior client deliverables, which felt encouraging.

Week 6

Sales and Service Cloud

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Sales Cloud features including leads, web-to-lead, lead conversion, accounts, contacts, account team, opportunity, opportunity team, products, price book and opportunity line items. Service Cloud features including cases, support process, case teams and queues, assignment and escalation rules, web-to-case and email-to-case, the Lightning service console, and Lightning Knowledge with data category groups. Also touched on duplicate management, multi-currency and multi-language.

2. Project Work Progress *(about 25 lines)*

With Sales and Service Cloud now in scope, I spent some time thinking about whether my UMS project should reuse standard objects or stick to fully custom objects. The trainer's recommendation - reuse what fits, customise what does not - made sense, but on close examination I decided that the admission lifecycle does not map cleanly to a Lead-to-Opportunity flow because applicants are not prospects in a sales pipeline but candidates being evaluated on merit. So I will keep Application as a custom object, while still using Contact as the shared person record across an applicant, student and alumnus lifetime. Cases will also not be the right model for student grievances in my project; I will revisit later. Lightning Knowledge gave me an idea, however - the structured data-category model used for knowledge articles is a good metaphor for how course materials should be organised, which feeds into the RAG design I am planning. I sketched the future Course Material object with a parent Course and a chunk-level metadata structure derived from this idea. I also built a small Experience Cloud sandbox community and configured guest user permissions just to feel how a public site behaves in Salesforce, which will be very relevant later.

3. Challenges Faced *(about 12 lines)*

I was tempted to misuse the Lead object for my student data because it had built-in conversion mechanics, but the trainer pushed back hard on that choice and explained why a custom object would be more maintainable in the long run. Lightning Knowledge with data category groups is a conceptually heavy area, particularly when knowledge needs to be filtered per audience in a console. Case assignment rules ordering and escalation timings took some practice to get right. The duplicate management settings are useful but the matching rule syntax took a while to grasp. The Experience Cloud guest user behaviour is significantly more restrictive than authenticated users.

4. Additional Comments *(about 15 lines)*

Sales Cloud is more thoughtful than I had imagined. The way the Lead-to-Account-to-Opportunity flow chains together, with each conversion preserving information, shows years of CRM thinking distilled into the platform. Service Cloud's knowledge model is the part I will most likely reuse conceptually, even if I do not use the actual Knowledge object itself. I have started thinking about the RAG assistant as a knowledge surface over course materials and faculty notes, which is essentially what Service Cloud's knowledge product is doing for support content. Experience Cloud surprised me with how much can be configured through audience-targeted theme settings without writing code; the platform's confidence in low-code is showing. Looking forward to next week's hackathon.

Week 7

Hackathon Phase 1, MFRP-1 and Experience Cloud

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Continued with Experience Cloud topics including audience targeting, themes, public versus authenticated pages and creation of communities. The major event of the week was the four-day Hackathon Phase 1, which mimicked an Agile Scrum framework with daily stand-ups, sprint planning, mid-sprint review and a final demo. Touched briefly on community licences, navigation and search. Closed with the first Mini Final Reflection Project submission.

2. Project Work Progress *(about 25 lines)*

This was the most intense and rewarding week so far. The hackathon ran across four days and our team of four was assigned the task of designing the first iteration of an end-to-end use case. We chose to build the foundation of a University Management System, modelled on a representative public university, and divided responsibilities so that one of us focused on the data model, two on the admission flow and one on the public-facing form. Day one was design - we finalised six custom objects (Department, Course, Faculty, Admission Round, Application, Application Document) and the relationships between them. Day two implemented the data model in our shared development org and added the record-triggered flow for acknowledgement emails along with validation rules for the application form. Day three built a first version of the public application form using a Salesforce-managed page in the Experience Cloud site, configured with restricted Guest User permissions so that an unauthenticated applicant could create exactly one Application record but read nothing else. Day four was integration, dry-run and demo. We presented to a panel of three Subject Matter Experts who graded us on data model cleanliness, workflow correctness, security posture and demo polish. We received positive feedback on our deliberate choice to keep the admission cell workflow declarative and to defer Apex until later.

3. Challenges Faced *(about 12 lines)*

The compressed timeline of a hackathon was the biggest pressure point. Four days felt short for the scope we had chosen, and we had to ruthlessly cut features that would not be demonstrable. Convincing the team on scope, particularly to leave out semester progression and result management for the first iteration, took a difficult half-hour discussion on day one. Configuring the Guest User profile correctly was harder than we expected - several iterations were needed before the unauthenticated form actually saved a record without exposing other applications. Coordination across four people in a shared org also required clear communication about who was editing what at any given time.

4. Additional Comments *(about 15 lines)*

This was the first time I worked in a real team on a project with a tight deadline and a panel review. The experience taught me as much about communication and division of labour as it did about Salesforce itself. Our team mentor, who joined us for thirty minutes each morning, was invaluable - his pushback on a couple of design choices saved us from going down dead ends. The MFRP-1 submission feels like a small but meaningful milestone. The judges' appreciation for the Guest User security audit specifically was very motivating, since that was something I had worked on personally for several hours. I am now confident that the University Management System will become my actual IIP project. I called my college mentor on Saturday and described the week, and her advice was to maintain the same intensity of design discipline as the work scales up.

Week 8

Interim Evaluation and Apex Basics

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

The first half of the week was the interim evaluation covering Days 1 through 17 of the curriculum, a comprehensive written and hands-on assessment. The second half began the Customisation block with Apex basics: data types and logic control, Apex classes, sObject relationships in code, SOQL and SOSL, Data Manipulation Language and Apex debugging. Also covered governor limits at an introductory level.

2. Project Work Progress *(about 25 lines)*

The interim evaluation took the first two days entirely. Once it was done and our scores were communicated, I returned to the UMS project for the remainder of the week. The Apex sessions opened up a new layer of the platform: I started rewriting one of the trickier declarative bits as Apex to feel the difference. Specifically, I wrote a first version of AdmissionFormController in Apex with a single AuraEnabled method that accepts a payload from the public form, validates it server-side, and inserts an Application record while setting status to Submitted. I used SOQL to look up the open Admission Round and populate it on the new application, and I used DML with explicit error handling rather than relying on Database insert with allOrNone semantics. The exercise made it clear how much of what I had been doing declaratively could also be expressed in Apex, and where the trade-offs lie. The trainer pushed us to write code with bulk safety from day one, so every Apex method I wrote accepted a list or a wrapper, never a single record. I committed the first Apex class into my project's Git repository on Friday, with a small unit test stub that I planned to fill out in the following week. My data model now has thirteen custom objects and the admission flow has its first piece of real backend code.

3. Challenges Faced *(about 12 lines)*

The interim evaluation itself was the immediate pressure. I had to revisit several topics, particularly OWD and sharing rules, the morning before the exam, and the assessment ran the full three hours without breaks. Once the assessment was over, switching mental modes from declarative to procedural code was a small but real shift. SOQL governor limits are an entirely new constraint compared to my college SQL experience - the idea that a query inside a loop can crash the transaction took a moment to internalise. The Apex syntax is close to Java but the platform's collection types and the AuraEnabled annotation are unfamiliar territory.

4. Additional Comments *(about 15 lines)*

Coding feels familiar after several weeks of clicks, and I noticed an immediate boost in my productivity once I returned to a code editor. The Apex IDE inside the developer console is acceptable but I have started using VS Code with the Salesforce extensions, which makes editing, deploying and running tests much smoother. The code review session with my industry mentor was the highlight of the week - he reviewed my AdmissionFormController class line by line and pointed out two issues, one around bulk safety and one around field-level security checks. I implemented both fixes the same evening. My evaluation score landed above the batch median, which gave me some confidence going into the next, heavier weeks.

Week 9

Apex Triggers, Tests and Visualforce

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Apex triggers in depth: trigger events, context variables, recursive triggers and how to prevent them with a static flag pattern, trigger order of execution, and an introduction to a basic trigger framework. Also studied exception handling, test classes and code coverage requirements. The Visualforce block was a quick tour through Visualforce pages, the MVC pattern, and the three flavours of Visualforce controllers: standard, custom and extensions.

2. Project Work Progress *(about 25 lines)*

With triggers now in my toolbox, I added a small piece of automation to the UMS project. I wrote a before-insert trigger on the Application object to enforce the Application Number format and to default the Admission Round lookup to the currently open admission round if the applicant submitted from the public form without setting it explicitly. I also wrote a corresponding test class with three test methods that achieved 92 per cent coverage on the trigger. The trigger framework pattern, where the trigger itself is a thin one-liner that delegates to a handler class, was something I copied from the trainer's reference code and applied to my own project. Visualforce was explored only briefly because LWC will arrive in a few weeks and the trainer made clear that for new development, LWC is the recommended path. I built a single sample Visualforce page just to feel the syntax, but I will not be using Visualforce in my actual project. I also took time this week to consolidate the project's metadata in source-format and pushed the entire thing to a fresh GitHub repository so that I have a versioned record of every change going forward. By Friday the project had thirteen custom objects, two Apex classes, one Apex trigger and a small set of test classes covering them.

3. Challenges Faced *(about 12 lines)*

My first encounter with a real recursive-trigger bug happened this week. An update fired by a trigger caused the trigger to fire again on the same record, and I had to introduce a static class variable as a recursion guard. Debugging through the developer console's log viewer was slow until I learnt to use filters effectively. Preparing test data inside test classes is verbose, and I have started to think about a small test data factory class that I can reuse across tests. The trigger order of execution diagram is dense and I keep a printed copy next to my keyboard.

4. Additional Comments *(about 15 lines)*

Triggers feel like the right place for cross-object integrity logic and I now understand why the trainer kept emphasising that triggers should be thin and delegate to handler classes. Test classes are starting to feel essential rather than ceremonial; the discipline of writing a test for every piece of code is something I want to carry forward into any framework I work with after this internship. The trigger framework class I copied this week is one of those small artefacts that I expect to keep reusing across projects. Deciding to skip Visualforce for the main project was a deliberate call, and it removed one layer of complexity from the design. The codebase is starting to feel like a real project rather than a series of training exercises.

Week 10

Asynchronous Apex and Deployment

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Synchronous versus asynchronous Apex with a focus on the future annotation, the Batchable interface, the Schedulable interface, Queueable Apex and the InvocableMethod annotation for use from flows. Studied the trade-offs across these patterns and when each is preferred. Introduction to change-set deployment between orgs and a quick recap of metadata versus source format. Closed with a full-day rapid holistic assessment.

2. Project Work Progress *(about 25 lines)*

Asynchronous Apex changed several design choices in the project. I re-architected the welcome-email step in the admission approval flow from a synchronous send to a Queueable so that the user-facing response remains fast even if the email service is slow. I also designed the seat-allocation engine as a Batchable Apex class so that it can iterate over an entire admission round's applications without hitting governor limits, and made the merit-score calculator a sibling Batchable with state passing through the Stateful interface. The retrieval-augmented-generation ingest, which I had been thinking about as a synchronous integration call, became a clear candidate for Queueable Apex because the outbound call to the Python service can be slow without affecting the faculty upload experience. By the end of the week I had three async Apex classes - WelcomeEmailQueueable, MeritEngineBatch and a stub for RAGIngestQueueable - in the repository and test classes covering each. The trainer's daylong RHA at the end of the week was a useful checkpoint and I scored well above my own expectations, which I attribute to the daily practice of small examples through the week.

3. Challenges Faced *(about 12 lines)*

Choosing between Batchable and Queueable was harder than I expected. Both can run asynchronously and both work for outbound callouts in their own ways, but the right choice depends on data volume and chaining requirements. I read the Salesforce documentation twice and ran a few benchmarks in my dev org before settling on the patterns I used. Test classes for async Apex have their own ceremony - calling Test.startTest() and Test.stopTest() around the async invocation - and I broke one of my tests by misordering these calls. Change-set deployment between scratch orgs is straightforward but slow, and I have started preferring sf project deploy commands from the SFDX CLI.

4. Additional Comments *(about 15 lines)*

The asynchronous pattern block was the most strategically useful week so far for my project. Several design choices that I had been deferring became clear once I understood the trade-offs. I now believe that any non-trivial Salesforce application has more async than synchronous code by line count, and that the architecture has to be planned with this in mind. The trainer's repeated reminder to put your slow work in Queueable and your bulk work in Batchable is a heuristic I expect to keep. The RHA on Friday felt smoother than I had feared. My industry mentor reviewed my updated project codebase on Friday evening and his only comment was to add a small bit of error handling around the welcome-email queueable, which I implemented immediately.

Week 11

Integration

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Integration introduction and patterns: point-to-point versus middleware approaches, fire-and-forget versus request-reply and the inbound and outbound variations. Covered SOAP and REST in detail with hands-on REST callout exercises, named credentials, external credentials and the principal model. Practised authentication patterns including OAuth 2.0 client credentials, JWT bearer flow and the simpler bearer-token approach.

2. Project Work Progress *(about 25 lines)*

This was the week where the RAG service architecture stopped being theoretical. I designed the end-to-end integration between Salesforce and a Python FastAPI service that will live on Render, with Qdrant managed cloud as the vector database. Two endpoints emerged from the design: a rag ingest endpoint for uploading a course-material PDF and a rag query endpoint for asking a natural-language question against a course-specific vector collection. On the Salesforce side, I created a named credential called RAG Service, attached an external credential with a bearer-token principal, and wrote a stubbed callout method inside RAGIngestQueueable execute that POSTs a base-64 encoded file payload to the Python service. I then started a separate small repository for the Python service and built a skeleton FastAPI application with the two endpoint stubs and the authentication middleware. By Friday the Salesforce side was ready to make calls and the Python side had a health endpoint responding to a curl request from my laptop, although the embedding and Qdrant write paths were still placeholders. I also read the original Retrieval-Augmented Generation paper over the weekend to ground the design in the actual technique rather than just the buzzword.

3. Challenges Faced *(about 12 lines)*

Salesforce's six-second synchronous callout timeout for triggers and the longer one-twenty-second limit for asynchronous callouts forced a clear design choice: all RAG ingest calls must be in Queueable Apex. The choice of authentication style took longer than I expected - I considered OAuth 2.0 client credentials briefly before settling on a simpler bearer token managed through the external credential principal. Apex's lack of support for binary multipart bodies was a quiet limitation; I worked around it by sending the file as a base-64-encoded string inside a JSON payload, which the Python side decodes. The named credential setup also requires careful permission set assignment for the calling user.

4. Additional Comments *(about 15 lines)*

The integration block tied together several earlier pieces - asynchronous Apex, security, error handling - into a coherent picture of how Salesforce talks to the rest of the world. The named credential and external credential abstraction is genuinely elegant compared to hardcoding URLs and secrets, and I do not understand why anyone would skip them even in small projects. Setting up the Python repository and pushing the first commit to GitHub felt like crossing into a different sub-project, but the discipline of a single design note covering both repositories has kept me organised. I am genuinely enjoying the project now and the late-evening hours on the Python side feel optional rather than mandatory.

Week 12

Modern JavaScript and JSON

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Deep coverage of ECMAScript 6 and later features: var versus let and const, arrow functions, object literals, constructor functions versus ES6 classes, callback functions, Promises, async and await, modules with import and export, destructuring of objects and arrays, the spread and rest operators, and modern array methods including map, filter, reduce, find and some.

2. Project Work Progress *(about 25 lines)*

With modern JavaScript fresh in mind, I returned to the Python service and added the ingest path properly. The Python side reads the incoming base-64 payload, decodes it into a PDF, chunks it using a paragraph-aware splitter, embeds the chunks with a sentence-transformer model and writes them into a per-course Qdrant collection. I also wrote the corresponding query path that embeds the query, retrieves the top-five most similar chunks and returns them. Although this is Python on the server, the JavaScript skills I refreshed this week will directly feed into the next two LWC weeks. I also took some personal time this week to prototype a small standalone HTML page with an embedded async-fetch call to my local Python service, simulating what the LWC will eventually do. The prototype confirmed that the Python service responds correctly under load when called concurrently from multiple browser tabs, with average response times of two to three seconds per query. I refined the system prompt for the assistant to be more disciplined about citations and to fall back gracefully when retrieval returns nothing useful.

3. Challenges Faced *(about 12 lines)*

Async and await took two full evenings of practice before I felt I had internalised it. The mental model of a promise as a value that will arrive later, rather than as a function to be called, was the conceptual shift I needed. Promise chains versus async-await is a stylistic choice but for any chain longer than two awaits I prefer the async-await form. Modules and import paths in modern JavaScript are deceptively simple in tutorials but require more care in real projects because of how bundlers handle them. JSON stringify with circular references caught me out once during the prototype work.

4. Additional Comments *(about 15 lines)*

JavaScript turned out to be much richer than I had thought from my undergraduate exposure. Several language features that exist in JavaScript - destructuring, spread, optional chaining - have no direct equivalent in the C-style languages I learnt at college, and they make code expressive in ways I had not previously appreciated. I am building a small running notebook of JavaScript idioms that I refer back to. The fact that the LWC framework relies heavily on modern JavaScript means that this investment will pay off concretely starting next week. The Python service is now functional enough that I can demo it end-to-end with a sample PDF and a sample question.

Week 13

Lightning Web Components - Foundations

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Introduction to the Lightning Web Component framework starting from Visual Studio Code with the Salesforce extensions, the Salesforce Developer Experience CLI, and the online code editors. Covered web stack background, the comparison between Aura and LWC, the component bundle structure (HTML, JavaScript, CSS, metadata), basic Salesforce DX commands, data binding from JavaScript into the template, the api, track and wire decorators at an introductory level.

2. Project Work Progress *(about 25 lines)*

This was the week I started building the user-facing layer of the project. I created the first real LWC, admissionForm, which renders a multi-section form (personal details, academic details, document upload, declaration) and calls AdmissionFormController submitApplication from Apex via an imperative call on the submit handler. I used the api decorator to expose two public properties so that the host page can pre-fill the form. CSS was scoped to the component using the shadow DOM defaults and I styled the form to feel consistent with the Salesforce Lightning Design System without pulling in heavy dependencies. By Friday the form was working end-to-end inside an internal App Builder page on my dev org, with all client-side validations and server-side error handling tested. I also created a small uploadCourseMaterial LWC in skeleton form for the faculty side, although its real wiring depends on the RAG ingest, which I have parked for next week. The LWC bundle structure feels clean and the local development cycle through SFDX is fast enough that I no longer dread small UI changes.

3. Challenges Faced *(about 12 lines)*

Shadow DOM CSS isolation was surprising at first - styles defined elsewhere on the page do not bleed in. This is a good default for component encapsulation but it took me a few iterations to realise why my page-level styles were not affecting the component. The trio of decorators - api, track and wire - is conceptually simple but easy to misuse; I used track unnecessarily on a primitive property and the trainer corrected me. SFDX deployment of metadata is significantly faster than change sets but has its own learning curve, particularly around source-format directory layout.

4. Additional Comments *(about 15 lines)*

Lightning Web Components are clearly the future of the platform; the framework feels modern and the gap from Aura is large. VS Code with the Salesforce extensions has replaced the developer console as my primary development environment, and the experience is now indistinguishable from a normal web frontend project. My morning git push has become a habit, and I now keep all my project work versioned across three branches: main, feature and a personal experiments branch. Building the admissionForm LWC was the first time I had something I could actually demo to a non-technical user, which made the work more rewarding. I showed the working form to my college mentor over a screen-share and she was visibly impressed.

Week 14

Lightning Web Components - Advanced

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Advanced LWC topics: JavaScript properties exposed across components, component lifecycle hooks including `connectedCallback`, `disconnectedCallback`, `renderedCallback` and `errorCallback`, component composition through parent-child nesting, events using `CustomEvent` for child-to-parent communication, the Lightning Message Service for cross-tree communication, accessing Salesforce data through the Lightning Data Service, the wire service against Apex, and the navigation service.

2. Project Work Progress *(about 25 lines)*

I built the `ragAssistantChat` LWC this week and wired it end-to-end with the Salesforce-side `RAGAssistantService` Apex class, which authorises a query and forwards it to the Python service via the named credential. The component renders a small chat surface with a query input, a streaming-style answer display and a citations list showing the source chunks with their page numbers. I used the wire service to fetch the current course's metadata and the imperative Apex call pattern for the query itself, which gave the right blend of reactivity and explicit control. I also created the `PortalRouterController` Apex class and the `portalRouter` LWC, which on `connectedCallback` detects whether the current user is a Student, Faculty or HOD and navigates them to the appropriate dashboard using the Lightning Navigation Service. By Friday the project had six named LWCs and four supporting Apex services. The customisation-block RHA went smoothly and my score landed close to the batch top.

3. Challenges Faced *(about 12 lines)*

The wire service's reactivity model needs careful thought; a wire adapter automatically refetches when its inputs change, which is normally desirable but can lead to surprise refetches if not understood. Imperative versus declarative Apex calls is a choice that depends on whether the data should be live-updating or fetched-on-demand, and I picked wrong the first time on the RAG query. Cross-component communication via `CustomEvent` worked, but the Lightning Message Service was the better choice for one of the wider broadcasts, and I refactored mid-week. The shadow DOM's testability is also an area where I want to spend more time.

4. Additional Comments *(about 15 lines)*

This week's exercises produced the first reusable component in the project - a small badge component that renders a status pill, used in three places - which felt like a small but important milestone. The wider data flow of the project is now clear in my head, and I can sketch it on a whiteboard quickly. The customisation-block RHA on Friday was the last big checkpoint before the consolidation week and project final stage. My industry mentor scheduled a longer review for next week to look at the full project end-to-end. I am beginning to think about how the final demo will be structured and what I want to highlight.

Week 15

Consolidation, AppExchange and MFRP Final Stage

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Consolidation day covering AppExchange and how managed packages are distributed, an overview of the latest Salesforce release notes and the cadence of three releases a year, recap and reinforcement of the customisation block topics, and pointers to advanced patterns that are common in production deployments. The MFRP Final Stage was formally launched with a project planning session and individual scope reviews.

2. Project Work Progress *(about 25 lines)*

The MFRP Final Stage launch shifted my focus formally from curriculum-led work to project-led work, although the two have been intertwined for several weeks. I spent the early part of the week revising the data model based on review feedback - splitting Faculty Course Mapping out from the Faculty object to support the case where a single faculty member teaches multiple courses across departments, and adding the Mentor Allocation object as a separate junction so that mentorship is tracked independently of course teaching. I refined three Apex services this week: ApplicationApprovalService now uses a savepoint correctly, MeritEngineBatch handles ties in merit by falling back to twelfth-standard percentage, and the SeatAllocationQueueable now respects the per-department per-category cutoff from custom metadata. The documentation also got an update with a fresh ER diagram and a sequence diagram of the admission flow. I reviewed the full codebase on Friday with my industry mentor for an hour, and we agreed on the priorities for the next four weeks: hardening the security model, RAG service polish and the Experience Cloud site build.

3. Challenges Faced *(about 12 lines)*

Balancing the consolidation revisions, AppExchange exploration and the deeper project work was tight on time. I spent some of my evenings reviewing old areas - particularly OWD and sharing rules - because the project's growing data model exposed gaps in my earlier mental model. Old data migration was a small but real problem when I refactored the Faculty-to-Course relationship; I had to write a one-off migration script to move data across the new structure without losing existing records. Mentor reviews are now mostly remote, and the lag between question and answer is occasionally a blocker.

4. Additional Comments *(about 15 lines)*

AppExchange is genuinely fascinating as an ecosystem; the fact that an entire layer of third-party functionality exists on top of the platform is not something I had appreciated before this week. I am informally thinking about whether parts of my UMS project, particularly the RAG service, could be packaged for distribution, although that is well beyond the scope of the IIP. The shift to MFRP-led work means most of my creative energy is now going into the project itself rather than into the curriculum. The trainer-facilitated sessions are lighter this week, which has been a welcome change in pace.

Week 16

Generative AI and Agentforce

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Generative AI in Salesforce: foundations of AI, predictive AI versus generative AI, key machine learning and deep learning concepts at a high level, AI capabilities in CRM, prompt engineering basics, introduction to Agentforce, the Einstein Trust Layer, the Atlas Reasoning Engine, the Prompt Builder with a flow-driven demo, agents with the reasoning engine, topics, instructions and actions, and a Data Cloud overview.

2. Project Work Progress *(about 25 lines)*

I evaluated whether to switch the project's AI assistant from the custom RAG service to Salesforce's native Agentforce, and after some deliberation I decided to keep the custom RAG. The reasoning was twofold: first, the custom service gives me a transparent retrieval step and a clear source-citation contract, both of which are important pedagogically and in the demo context; second, Agentforce currently requires a higher edition of Salesforce than my Developer Edition can demonstrate end-to-end. That said, the Prompt Builder concepts directly improved my custom system prompt, and I rewrote the assistant's prompt to be more disciplined about citing chunks, declining when retrieval returns nothing relevant, and adopting a teacher-like rather than encyclopedic voice. I also built the AI Chat History object that records every assistant interaction (role, course, query text, response text, list of source chunks, response time and an error flag), and wired the `ragAssistantChat` LWC to insert a row into this object after each call. I demoed the assistant end-to-end to two SMEs in a half-hour informal session and received useful design feedback.

3. Challenges Faced *(about 12 lines)*

Prompt design turned out to be more subtle than I expected. Several phrasings that I thought were neutral produced biased or vague answers, and I had to iterate on the system prompt at least five times before it behaved consistently. Hallucination handling - what should the assistant do when the retrieved chunks are not relevant enough - is a real design choice and I settled on an I am not sure based on the course material fallback. The decision between native Agentforce and custom RAG took up the better part of two days of deliberation including a long discussion with my mentor. The Einstein Trust Layer is impressive but requires a higher edition.

4. Additional Comments *(about 15 lines)*

The Salesforce AI ecosystem is moving faster than I had assumed, and the Agentforce demo specifically was the most visually impressive thing I have seen all programme. The architectural patterns are similar to what I am building, just more polished and integrated. I noted several ideas to incorporate into my own assistant. The hackathon extension - where we were asked to add a simple Agentforce-style touch to our project - was a useful nudge to actually deploy and test the custom RAG end-to-end rather than keep treating it as a future task. I am increasingly confident that the assistant is going to be the most visible feature of the final demo.

Week 17

DevOps

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

DevOps fundamentals applied to Salesforce: version control with Git, branching strategies, pull-request workflows and code review, Salesforce DX with the sf command-line interface, continuous integration and continuous deployment pipelines, automated Apex testing, code scanning for security and quality, deployment strategies including scratch orgs and feature branches, and basic monitoring practices.

2. Project Work Progress *(about 25 lines)*

I set up the Salesforce side of the project to follow modern DevOps practices. The repository now lives on GitHub with main, develop and feature branches, and every push opens a pull request to develop. I configured a GitHub Actions workflow that, on every pull request, authenticates to a scratch org using a JWT bearer flow, deploys the source, runs the full Apex test suite and reports the result back to the pull request. The Python RAG service repository was similarly configured with a Render auto-deploy hook tied to the main branch, so a push automatically rebuilds and redeploys the service. I added a small README to both repositories with the local setup steps and a description of the integration points. By Friday I had merged seven feature branches into develop, including the refinements from the previous two weeks, and the develop branch was clean and tested. I also added a code-scanning step using a free SAST tool to flag obvious security issues in the Apex code.

3. Challenges Faced *(about 12 lines)*

Merge conflict resolution in a shared repository (even with just my own branches) is fiddly when refactors touch the same files; I had two non-trivial conflicts this week that took half an hour each to resolve carefully. Scratch org configuration through the project scratch definition JSON file required some trial and error - I had to enable specific features and feature licences that my org uses in production. CI workflow YAML syntax is dense, and a couple of my workflow runs failed for indentation reasons that I had to chase down. Render's free-tier cold-start was a documented limitation that I learnt to work around with a small UptimeRobot ping every ten minutes during demo hours.

4. Additional Comments *(about 15 lines)*

The Git workflow feels natural now, and the discipline of writing a one-line commit message that summarises a single coherent change has noticeably improved the readability of the project's history. Pull requests passing CI checks before merge is the single biggest quality improvement of the week, and it is something I will carry forward into any future project. Watching the pipeline run against my pull request and seeing all green tests is a small but real satisfaction. I feel for the first time like a real industry developer rather than a student practising on Salesforce.

Week 18

Project Phases 1 and 2 - Master Objects and Faculty

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Customisation work continued on a project-by-project basis rather than via formal training. Skills sharpened this week: profile cloning and the precise differences between profile-level and permission-set-level controls, custom permission definition and checking via FeatureManagement, user provisioning through the System Administrator profile, quick action setup on both list and record contexts, and Lightning Record Page customisation.

2. Project Work Progress *(about 25 lines)*

This was the first formally project-only week, with no training sessions on the schedule. I focused on the foundations: Phase 1 (master objects) and Phase 2 (faculty). I finalised the Department object with the four required fields including HOD User, and the Course object with semester, credits, type and capacity fields. I seeded the system with five real departments modelled on Shivaji University's existing structure and ten representative courses across them. For Phase 2, I cloned the Standard User profile into a new Faculty profile and applied selective object and field permissions. The HOD Permission Set was created and assigned to one user per department, while the custom permission Can Approve Semester Progression was attached to the permission set rather than to the profile. I provisioned two faculty users per department (ten total) and verified the head-of-department designation by logging in as each. I also configured page layouts for the Faculty profile so that the irrelevant standard tabs were hidden from their app launcher. Documentation was updated with a fresh user-role matrix and an actor diagram showing who can see and do what.

3. Challenges Faced *(about 12 lines)*

The choice between cloning a profile versus building everything on the Standard User profile plus permission sets came back, and I deliberated again this week before settling on a cloned Faculty profile as the base. Custom permission scoping took more care than I had expected - the FeatureManagement checkPermission call requires the exact developer name and I had a typo that took me twenty minutes to find. Lookup versus master-detail came up again on the Faculty-to-Department relationship, and I went with lookup so that a faculty member's department can be changed without data loss.

4. Additional Comments *(about 15 lines)*

A backend-first design is paying off; the absence of any user-interface work this week meant that I could focus entirely on the data model and the security posture without worrying about how things look. Documentation kept current alongside the implementation is another habit that pays off - my industry mentor reviewed the user-role matrix on Friday and was able to give meaningful feedback in fifteen minutes rather than asking me to walk him through the project. My confidence in the project is growing each week.

Week 19

Project Phases 3 and 4 - Admission Application and Admission Cell

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Project skills further sharpened: Guest User profile configuration with least-privilege access, configuration of a public Experience Cloud page with audience targeting, custom metadata types for administrator-editable configuration including Merit Config and Department Cutoff, the wire service and AuraEnabled annotations for LWC-Apex bridging, and Batchable Apex with stateful processing.

2. Project Work Progress *(about 25 lines)*

Phases 3 and 4 - the public admission application and the admission cell workflow - landed this week. The admissionForm LWC was published to a public Experience Cloud page after a careful Guest User profile audit (create on Application, no read on anything else, no access to Student, Faculty or User). The form was tested end-to-end with five sample applications, each submitted from an unauthenticated browser session, and each producing a record with the correct status and a confirmation email. The verification quick action was added to the Application page layout for the Admission Cell profile, with the quick action invoking a screen flow that lets the cell mark each document as verified or flagged. The discrepancy email is sent via a Lightning Email Template through a record-triggered flow. The MeritEngineBatch was tested against the five sample applications and correctly populated the Merit Score field using the weights from the custom metadata. The SeatAllocationQueueable was triggered from the batch's finish method and correctly ordered applications by merit and allocated up to the per-department capacity.

3. Challenges Faced *(about 12 lines)*

Guest User security took several iterations to get right. The first version of the Guest User profile gave too much read access on related objects, which the Login As exercise caught. The Application status transitions need a state-machine guard because a careless flow could move an application from Submitted to Approved without verification; I added a validation rule that explicitly enforces the allowed transitions. The discrepancy email flow had a subtle bug where the email template referenced a field that was empty on draft applications, and the flow failed silently for those cases until I added a null guard.

4. Additional Comments *(about 15 lines)*

The admission flow end-to-end works, and seeing a sample applicant submit a form from a completely separate browser, receive an acknowledgement email, and then have the admission cell verify the documents inside the org - all without me being involved in between - was the most satisfying single moment of the project so far. Security tightening was an iterative process rather than a one-shot, and the Login As pattern saved me at least three potential issues. One genuine bug was found through this method and I added it to a small log of design lessons that I will share in the final report.

Week 20

Project Phases 5 and 6 - Student Conversion and RAG Integration

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Project-only skills sharpened: atomic Apex transactions with Database setSavepoint and Database rollback for rollback under exception, configuration of External Credentials and principal-based authentication, Python service deployment on Render with environment variables for secrets, Qdrant vector database collection management, embedding model selection and chunking strategy choices.

2. Project Work Progress *(about 25 lines)*

Phases 5 and 6 - student conversion and RAG integration - completed this week. The ApplicationApprovalService approve method now atomically creates a Contact under a shared Shivaji University Students account, inserts the Student record, creates the User record using the Student profile, links the user back to the student, updates the application status to Admission Approved and enqueues the welcome email. I tested the rollback path by deliberately failing the user-creation step and confirmed that the application and the contact were both reverted. The RAG service was the bigger lift: the Python rag ingest endpoint now accepts a base-64 encoded PDF, chunks it using a paragraph-aware splitter, embeds the chunks with a sentence-transformer model running locally on the Render dyno, and writes them into a per-course Qdrant collection. The rag query endpoint embeds the query, retrieves the top-five chunks and constructs a role-aware system prompt before invoking the LLM. I deployed both endpoints to Render with an environment-variable bearer token, configured the Salesforce External Credential to use the same token, and tested the round-trip from the faculty's upload through to a student's query. By Friday the assistant was answering questions correctly with page-level citations.

3. Challenges Faced *(about 12 lines)*

Savepoint and rollback testing required deliberate failure injection, which is something I had not practised before in Apex. I added a small TestVisible boolean inside ApplicationApprovalService that forces a synthetic failure during tests. Render's free-tier cold-start (the service scales to zero after fifteen minutes of inactivity) was the most annoying real-world issue and added thirty to fifty seconds to the first query of a session; I worked around it with UptimeRobot. Bearer token rotation required me to update both Render and the Salesforce External Credential in lockstep, and I documented the steps so that the next rotation is faster.

4. Additional Comments *(about 15 lines)*

Watching the assistant answer a real question from a real PDF, with actual page numbers in the citations, was the project's biggest milestone for me personally. The end-to-end loop - faculty uploads a course material, Queueable forwards it to Python, Qdrant indexes it, student queries, Python retrieves and synthesises, citation returned - is now smooth enough that I can demo it to anyone with three or four sample questions and feel confident. The cold-start mitigation is a small hack but it makes the demo experience reliable enough.

Week 21

Project Phase 7 and Final Evaluation

1. Notes about Equipment, Software and New Skills Learned *(about 6 lines)*

Final-week skills: Experience Cloud Lightning Web Runtime site configuration end-to-end, audience targeting for role-based page access, demo scripting and presentation craft, IIP report writing in LaTeX, plagiarism checking and similarity reduction techniques.

2. Project Work Progress *(about 25 lines)*

The final project phase was the Experience Cloud Lightning Web Runtime site. The site has a public homepage, a public Apply page hosting the admissionForm LWC, the Salesforce-managed login page, a post-login portal router page and three role-specific dashboard pages for Student, Faculty and HOD. The portalRouter component on connectedCallback invokes PortalRouterController detectUserRole which returns one of student, faculty, hod or unknown. Each dashboard is restricted using Experience Cloud audience targeting so that a user who manually navigates to a dashboard they are not entitled to is denied access. End-to-end user acceptance testing was run on Tuesday and Wednesday with my mentor playing the role of an applicant, a student and a head of department, and several small issues were found and fixed. The final demo was delivered on Thursday to a panel of three SMEs and one client representative, and the feedback was overwhelmingly positive on the security posture, the RAG assistant's grounded answers and the overall coherence of the design. The IIP report was finalised on Friday and submitted along with a plagiarism similarity check below the fifteen per cent threshold.

3. Challenges Faced *(about 12 lines)*

The Lightning Web Runtime template's constraints around custom CSS and page layouts required a few workarounds, particularly for the post-login portal router where I wanted a centred loading spinner. Audience targeting verification was a careful process - I had to confirm not only that authorised users see their dashboard but also that unauthorised users see an explicit denial rather than an accidental render. Demo preparation nerves were real, and I rehearsed the full demo three times with different sample data on Wednesday. The IIP report writing took longer than I had estimated.

4. Additional Comments *(about 15 lines)*

The project is complete, the demo went well, and the IIP report is submitted. Looking back at the twenty-one weeks, the journey from a college student who had never opened a Salesforce org to someone who has built a non-trivial multi-role application with a custom AI integration is a real arc. The Cognizant Gen C programme is intense but rewarding, and the project itself is something I expect to continue working on in spare time after the internship ends. My industry mentor, my college mentor and my HOD have all been crucial to making this work, and I will be thanking them in person at the institute's IIP presentation next month. The final demo standing ovation will stay with me for a long time.